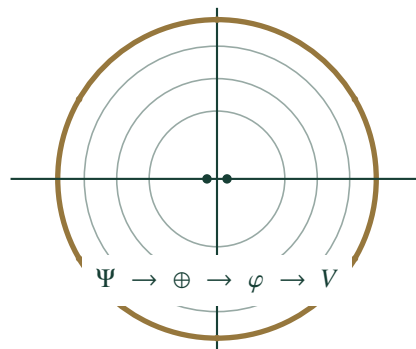


atomOS

Mirage / Ring Edition

Mathematical specification &
texture-programmed CUDA kernels

RTX 5070 Ti target / compute capability 12.0



Tom Klootwijk

NL200678942 • 10-07-1990

Native execution; texture-addressed rules. A CUDA interpreter reads a programmable transition table through texture objects. A second kernel performs identity-seeded propagation with explicit shift/XOR/hinge precedence, RK4 integration and binary boundary tests.

13 September 2026. Author attribution as supplied. CPU execution is tested; CUDA source is supplied for local compilation and device validation. The package is not an installation on the author's hardware.

Edition contract and reading guide

This edition formalizes the engine as a computational object, keeps the source’s vocabulary, and makes its operators type-correct. The requested “kernel in texture” becomes a precise architecture: **native CUDA code interprets a transition program whose words are fetched as texture data**. “Universal” means programmable computation with the memory assumptions stated in Section 12; it does not mean that a hash or a coordinate system supplies every physical law.

Three executable profiles. The propagation profile, atomOS-P, maps identity-seeded states through the geometric and Boolean cascade. The literal word profile, atomOS-W, retains the source’s shift/merge/XOR register equation without confusing it with a tree address. The programmable profile, atomOS-U, adds explicit machine state and a writable binary tape; its transition table is the texture-addressed program. The two runnable demonstration modes are propagation and universal interpretation. The separate CUDA word primitive is also included in source.

Source and correction notation. S0 is the earlier 42-page transcript, S1 the latest 42-page transcript, and S2 the preceding formalization. Their full digests identify them in the source manifest. C01–C14 mark mathematical corrections. D01–D04 mark added implementation choices or boundaries. The source’s author prompts and generated responses are not conflated: reported outcomes such as instantaneous prediction or external access are not treated as observations merely because they occur in the transcript.

Hardware contract. The RTX 5070 Ti target is NVIDIA compute capability 12.0; CUDA 12.8 introduced compiler support for SM_120. [N1–2] The delivered source uses CUDA texture objects, not a cache-pinning mechanism. The host process remains in user mode. No custom ring-0 driver, external control connection, neural interface or physical enforcement mechanism is included.

Execution record. The portable C++ engine, Python reference and supplied tests were run in the delivery environment. CUDA was not compiled or executed there because nvcc and an accessible NVIDIA GPU were unavailable. Sections 19 and 20 separate completed checks from the on-device acceptance procedure. All demonstration data are synthetic.

Navigation. Sections 1–10 give the corrected engine; 11–13 establish invariants and programmability; 14–16 give the GPU and privilege design; 17–20 cover presentation, complexity, validation and operation. The closing correction register and references provide page-level traceability. Editable LaTeX, C++/CUDA/Python source, tests, sample records and an offline viewer accompany this PDF.

Attribution. Tom Klootwijk, NL200678942, 10-07-1990 are reproduced as requested. They are not a password, cryptographic signature, credential or assertion of access to another system. This specification and implementation were prepared with AI assistance.

Contents

Edition contract and reading guide	1
1 Source structure and retained engine	3
2 Typed state space and the double-dot nucleus	4
3 Identity, seed placement and reproducible jitter	5
4 L-system, bit shifts and Hadamard-labelled blend	6
5 Lower-case φ and the log-polar chart	7
6 Kinematic calculus and environmental divergence	8
7 Klein quotient, V mirror and orientation	9
8 RK4 integration and its error budget	10
9 One-bit word definitions and the POPCNT gate	11
10 Complete operator-precedence transition	12
11 Invariants and provenance	13
12 The texture-programmed universal profile	14
13 Constructive simulation and universality	15
14 RTX 5070 Ti: what is in texture memory	16
15 GPU memory layout, lifetime and launch design	17
16 Ring 0, access authority and deployment boundaries	18
17 Bush mirage, the ring and the JK operator	19
18 Complexity, observables and benchmarking	20
19 Executed validation and pending GPU acceptance	21
20 Build, run, inspect and reproduce	22
21 Correction register and implementation boundaries	23
Package inventory and source provenance	24
Official technical references	25

1 Source structure and retained engine

The source builds from cloud-chamber track formation to a seed-based L-system; introduces double UU identity and a log-polar LUT; adds RK4, Klein wrapping and packed masks; and then extends its language to wider applications. This sequence, rather than an unrelated generic simulator, organizes the formalization. [S1, pp. 3–31]

S1 pages	Source element	Retained mathematical role
3–4	BBV; P/B hinge; V gestation	Basis/orientation change; state-to-observation staging.
4–9	Hadamard blend; Ψ ; φ ; 0-order L-system	Boolean XOR, typed phase, context-free branching with explicit lineage.
11–13	Double UU ID; double-dot nucleus	Carrier descriptor paired with a full 256-bit identity digest.
13–18	V mirror; RK4; log-polar LUT; $\theta \equiv \varphi$	Defined field integration on a two-coordinate chart; conditional mirror symmetry.
18–24	Hash-to-coordinate map; environment; divergence	Deterministic synthetic placement; separate model fields and coordinate conversion.
25–31	One-bit SDF words; POPCNT; Vantablack	Packed membership predicates, same-domain intersections and absorbing active state.
31–42	External/neural/physical-control narrative; JK	No such external interface is established or implemented. JK receives its Boolean truth table.

1.1 What the diagrams determine

The packed-coordinate diagram on S1 page 16 assigns two 16-bit fields to radial and angular indices. That is a *32-bit coordinate key*, not a single Boolean occupancy value. The operator-flow diagram on page 30 fixes shift before XOR before mask resolution. These visual relationships are preserved, with geometry inserted as the explicitly defined address-resolution stage before spatial mask lookup. [S1, pp. 16, 30]

S1 pages 6 and 9 use both XOR and OR for merging shifted words. Their results differ when shifted bits overlap. This edition exposes both word semantics rather than silently claiming they coincide. The geometric propagation profile uses distinct child identifiers and a separate one-bit jitter veto. [C03–C04]

1.2 Operational thesis

The engine is a reusable transition architecture: identity chooses a reproducible initial state and jitter sequence; a branching grammar enumerates candidate states; geometry assigns candidate locations; environment predicates accept or absorb them; and a journal binds the execution sequence to its inputs. A domain-specific field and observation model must be explicit inputs to that architecture, not hidden implications of its names.

2 Typed state space and the double-dot nucleus

Let $\mathbb{B} = \{0, 1\}$, $W_{32} = \{0, \dots, 2^{32} - 1\}$, and $H_{256} = \mathbb{B}^{256}$. Let C be the set of carrier descriptors. Define the double UU envelope by

$$UU = (c, d) \in C \times H_{256}, \quad d = \text{SHA256}(\text{encode}(c)). \quad (1)$$

The pairing is a typed product. It is not ordinary set union: $U \cup U = U$ would not preserve two distinguishable layers. The double-dot nucleus denotes the binding of the carrier description and its recorded identity. [S1, pp. 11–13] [C05]

The propagation state at generation n is

$$S_n = (d, n, F_n, E, \vartheta, \ell_n), \quad (2)$$

where F_n is the live frontier, E the immutable environment snapshot, ϑ all rules and parameters, and ℓ_n the journal head. Each node is

$$x = (h, \rho, \varphi, a, q, \sigma, o). \quad (3)$$

Here h is a lineage identifier, (ρ, φ) a chart coordinate, $a \in \mathbb{B}$ the active bit, q the resolved occupancy-cell address, σ a status/reason, and $o \in \mathbb{B}$ a local orientation parity.

Symbol	Type	Meaning
Ψ or a	Boolean active state	Not a coordinate, digest or quantum amplitude.
Ψ_W	W_{32}	Register in the separate literal word profile.
φ	Angle modulo 2π	Lower-case hinge; distinct from the golden ratio.
ρ	Real chart coordinate	$\ln(r/r_*)$ on the annulus; abstract quotient coordinate in Klein mode.
q	Integer cell address	Obtained by quantization; not itself an occupancy mask.
E_q	Boolean	Occupied/blocked cell predicate.
V	Environment plus observation stage	The source's gestation/womb name.
$\Delta\Delta T$	Positive generation interval	Implemented as Δt , subdivided numerically.

Kernel interface. $\mathcal{K}_\vartheta : (S_n, I_n, E_n) \mapsto (S_{n+1}, O_n, \ell_{n+1})$. Input, state, observation and provenance are separate types. Hashing a descriptor does not measure its location or authorize operations on it. The synthetic implementation uses a text seed, not a bank account or an individual subject.

3 Identity, seed placement and reproducible jitter

3.1 The full digest remains present

A single Boolean may answer a predicate about a digest, but cannot encode every possible digest injectively. If $f : H_{256} \rightarrow \mathbb{B}$ were injective, its image would contain 2^{256} distinct elements inside a two-element set, a contradiction. A coordinate key with 32 bits is also a many-to-one projection of the 256-bit space. This corrects the source's one-bit compaction language without discarding its packed-predicate objective. [S1, pp. 12, 18–19, 28] [C06]

The host implements SHA-256 and retains all 32 digest bytes. It is tested against known digest vectors and Python's independent implementation; the standard algorithm is specified in NIST FIPS 180-4. [N9] The digest alone is not a secret or a digital signature.

3.2 Hash-to-log-polar placement

Split d into two 128-bit halves. Extract their leading 16 bits as $u, v \in \{0, \dots, 65535\}$. The implemented seed projection is

$$\rho_0 = \rho_{\min} + (\rho_{\max} - \rho_{\min}) \frac{u + 1/2}{65536}, \quad (4)$$

$$\varphi_0 = 2\pi \frac{v + 1/2}{65536}, \quad h_0 = 1, \quad a_0 = 1. \quad (5)$$

Midpoint placement avoids exact outer boundaries in the mathematical model. The complete digest is still the identity; u, v only choose synthetic coordinates. The root digest does not encode a geographic position or prove a relationship among financial routing, particle motion and biological transmission. [C06–C07]

3.3 One-bit jitter log

For candidate child h_b at generation n , define

$$j_{n,h_b} = \mathbf{1}[\text{byte}_0 \text{SHA256}(D_J \| d \| \text{LE64}(n) \| \text{LE64}(h_b)) < \tau_J], \quad (6)$$

where D_J is the UTF-8 string `atomOS:jitter:v2` followed by a zero byte, and $\tau_J \in \{0, \dots, 256\}$. The default threshold is 32. Under a uniform-byte model its veto probability is $32/256 = 1/8$; for a fixed seed the realized sequence is deterministic, not a fresh physical measurement.

The host computes these bits before each launch and packs candidate k at word $\lfloor k/32 \rfloor$, bit $k \bmod 32$. Thus candidate placement and jitter remain identity-bound, while the GPU consumes an immutable jitter snapshot. Recomputing a hash after changing the input does not preserve the low-order angle by mathematical necessity. Continuity must come from a separate stable identifier or explicit tracking model. [C06]

4 L-system, bit shifts and Hadamard-labelled blend

4.1 Context-free grammar and lineage

Use the retained grammar

$$0 \longrightarrow 0, \quad 1 \longrightarrow 1[\varphi_+\Psi] \ 1[\varphi_-\Psi]. \quad (7)$$

The grammar determines branching syntax. The execution state carries identifiers and geometry. A rooted binary lineage tree is not automatically a binary *search* tree: no key-ordering invariant is specified in the source. [S1, pp. 6–9] [C03]

For side $b \in \{0, 1\}$, assign

$$h_b = 2h + b = (h \ll 1) \mid b, \quad \text{parent}(h_b) = h_b \gg 1. \quad (8)$$

The right child is not $h \gg 1$. For example, the two children of seed 1 are 2 and 3, while $1 \gg 1 = 0$. Unsigned 64-bit overflow is detected before creating a generation. A parent must satisfy $1 \leq h \leq 2^{63} - 1$; generation 63 is the deepest representable full-width lineage level.

4.2 The circled-plus operator

For Boolean a, j , the retained blend is XOR:

$$a \oplus j = a + j \pmod{2}. \quad (9)$$

Its truth table is $0 \oplus 0 = 0$, $0 \oplus 1 = 1$, $1 \oplus 0 = 1$, and $1 \oplus 1 = 0$. Elementwise multiplication and the quantum Hadamard matrix

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$$

are different operators. The name ‘‘Hadamard blend’’ is retained as source vocabulary; the code uses XOR, not a quantum gate or a noise-removal theorem. [C02]

For a live parent’s child, $a_b = 1 \oplus j_{n,h_b}$. An already absorbed parent never enters this rule. Otherwise an unconditional XOR could create a new 1 from a dead 0, contradicting the absorbing-state requirement.

4.3 Literal word profile, kept separate

For unsigned word w , jitter word J and same-domain mask M , set

$$s_{\oplus} = (w \ll 1) \oplus (w \gg 1), \quad s_{\vee} = (w \ll 1) \vee (w \gg 1), \quad (10)$$

$$z = s_* \oplus J, \quad W(w, J, M) = \begin{cases} 0, & w = 0 \text{ or } z \wedge M \neq 0, \\ z, & \text{otherwise.} \end{cases} \quad (11)$$

All arithmetic is explicitly 32-bit unsigned. XOR is the default merge; OR is an alternative. For $w = 5$, the XOR result before jitter is 8 and the OR result is 10. This resolves the conflicting source equations visibly. [C04]

5 Lower-case φ and the log-polar chart

5.1 A genuine two-coordinate domain

On an annulus with reference length $r_* > 0$, define

$$\rho = \ln(r/r_*), \quad x = r_* e^\rho \cos \varphi, \quad y = r_* e^\rho \sin \varphi. \quad (12)$$

The chart has two degrees of freedom. Writing $\theta \equiv \varphi$ chooses a reduced model; it does not give a lossless, invertible representation of arbitrary three-dimensional position. Nor does a two-entry column vector make the spatial state one-dimensional. A genuine 3-D model must retain a second independent angular coordinate or impose a specified geometric constraint. [S1, pp. 15–18] [C07]

The metric and area element are

$$ds^2 = r_*^2 e^{2\rho} (d\rho^2 + d\varphi^2), \quad dA = r_*^2 e^{2\rho} d\rho d\varphi. \quad (13)$$

These follow by differentiating x, y and expanding $dx^2 + dy^2$. Uniform log-radial spacing gives finer physical spacing near the inner radius and larger cells farther out. The chart excludes $r = 0$, where $\ln r$ is undefined. “Apex” names the seed/reference construction, not an exact representation of the excluded origin.

5.2 Hinge and BBV orientation

For a node with orientation parity o , side b applies

$$\varphi_b = \text{wrap}_{2\pi} \left(\varphi + (-1)^{b+o} \Delta\varphi \right), \quad \Delta\varphi = \frac{2\pi k_\varphi}{N_\varphi}. \quad (14)$$

The angle index changes by a configured step; that is modular addition, not rotation of the binary encoding of an integer. Bitwise rotate and angular addition coincide only under a separately specified representation. [C07]

The BBV “P-to-B” hinge is implemented as an orientation change of a state or basis. A rotation $R(\varphi)$ preserves vector length, with $R^T R = I$. It does not turn momentum units into magnetic-field units, clone a physical eigenvector, or by itself specify a force law. A physical charged-particle adapter would need its own position/momentum equations and input fields; the supplied field is synthetic. [S1, pp. 3–4] [C01]

5.3 What the LUT stores

The angle table stores pairs $(\cos \varphi_j, \sin \varphi_j)$ at N_φ evenly spaced samples. Trigonometry occurs during host table construction, not at each GPU field evaluation. Symmetric nearest-angle lookup uses an even cosine extension and odd sine extension; zero and half-turn sine values are set to zero explicitly. The table still uses floating-point values and multiple bytes per sample: one-bit storage applies to predicates, not to every field.

6 Kinematic calculus and environmental divergence

Let $\mathbf{u} = u_r \mathbf{e}_r + u_\varphi \mathbf{e}_\varphi$ be a planar velocity field expressed in the *orthonormal* polar basis. Along a trajectory,

$$\dot{\rho} = \frac{u_r}{r_* e^\rho}, \quad \dot{\varphi} = \frac{u_\varphi}{r_* e^\rho}. \quad (15)$$

The factor $e^{-\rho}/r_*$ converts physical velocity to coordinate rate. It is not an automatic decay law for the physical velocity. In particular, a coordinate transformation cannot turn constant physical motion into attenuation of its physical speed. [S1, pp. 23–24] [C10]

Starting from planar polar divergence,

$$\nabla \cdot \mathbf{u} = \frac{1}{r} \frac{\partial(r u_r)}{\partial r} + \frac{1}{r} \frac{\partial u_\varphi}{\partial \varphi} \quad (16)$$

$$= \frac{e^{-\rho}}{r_*} \left(\frac{\partial u_r}{\partial \rho} + u_r + \frac{\partial u_\varphi}{\partial \varphi} \right). \quad (17)$$

Thus the source expression is valid when its component symbols mean orthonormal physical components and $r_* = 1$. With coordinate-rate components $v^\rho = \dot{\rho}$, $v^\varphi = \dot{\varphi}$, the corresponding expression is instead

$$\nabla \cdot \mathbf{u} = \partial_\rho v^\rho + 2v^\rho + \partial_\varphi v^\varphi. \quad (18)$$

This follows by substituting $u_r = r v^\rho$, $u_\varphi = r v^\varphi$. The coefficient convention must therefore be declared.

6.1 Implemented annular field

In the dimensionless synthetic profile, $r_* = 1$ and

$$f_\rho(\rho, \varphi) = g + e^{-\rho}(w_x \cos \varphi + w_y \sin \varphi), \quad (19)$$

$$f_\varphi(\rho, \varphi) = \omega + e^{-\rho}(-w_x \sin \varphi + w_y \cos \varphi). \quad (20)$$

It combines radial growth, angular drift and a Cartesian drift vector. The default coefficients are $g = 0.3$, $\omega = 0.45$, $w_x = 0.08$, $w_y = -0.04$. These are implementation parameters, not calibrated environmental measurements. [D01]

A uniform Cartesian drift has zero divergence: its polar components satisfy $u_r = w_x \cos \varphi + w_y \sin \varphi$ and $\partial_\varphi u_\varphi = -u_r$. This is a useful check against interpreting every phase change as expansion. An absorbing mask sets active state to zero; it does not entail a physical divergence of negative infinity. Likewise, a geometric contact predicate alone is not a measurement of infection, dose, financial default or any other domain outcome.

7 Klein quotient, V mirror and orientation

The optional Klein mode is an abstract quotient chart, not a globally embedded volume with a conventional inside/outside sign. Let $L = \rho_{\max} - \rho_{\min}$ and impose the identifications

$$(\rho, \varphi) \sim (\rho, \varphi + 2\pi), \quad (\rho, \varphi) \sim (\rho + L, -\varphi). \quad (21)$$

The first seam is periodic and the second reverses orientation. The fundamental domain is $[\rho_{\min}, \rho_{\max}) \times [0, 2\pi)$. [S1, pp. 5, 7–9, 17] [C08]

For an unwrapped state, define

$$q_w = \left\lfloor \frac{\rho - \rho_{\min}}{L} \right\rfloor, \quad (22)$$

$$\rho' = \rho - q_w L, \quad (23)$$

$$\varphi' = \text{wrap}_{2\pi}((-1)^{q_w} \varphi), \quad o' = o \oplus (q_w \bmod 2). \quad (24)$$

Negative wraps obey the same floor/parity rule; two wraps restore orientation. Tangent components transform with Jacobian $\text{diag}(1, -1)$ on an odd wrap. The parity bit preserves the meaning of the left/right hinge across a reflected seam.

7.1 A field compatible with the quotient

A vector field must respect the seam identification. The Klein demonstration therefore does not blindly reuse the annular radial field. It uses

$$\dot{\rho} = g + w_x \cos \varphi, \quad \dot{\varphi} = \omega \sin \varphi + w_y \sin(2\varphi). \quad (25)$$

The radial component is even and the angular component odd in φ , giving the required transformation across the reversed seam. The angle LUT uses the same even/odd construction. Finite precision remains a numerical approximation to that quotient model.

7.2 V and the one-bit boundary

The environment is a separately defined predicate $E : K \rightarrow \mathbb{B}$ on canonical chart cells. It can represent blocked cells on the quotient. It is not a global signed-distance function of the Klein bottle embedded without intersections in ordinary 3-D space. “One-bit SDF” is therefore retained as a name for a thresholded occupancy predicate, not a claim to preserve distance magnitudes or a global inside/outside orientation. [C08–C11]

Mirror contract. Equal-and-opposite hinge steps do not guarantee equal outcomes. Mirror symmetry of complete trajectories requires symmetric initial conditions, compatible fields, symmetric masks, matching numerical treatment and a correlated jitter construction. The source’s “SSymmetry” names this conditional mirror relation; it is not a proof of particle-physics supersymmetry or quantum interference.

8 RK4 integration and its error budget

For a specified differential equation $\dot{y} = f(t, y)$ and step $h > 0$, classical RK4 is

$$k_1 = f(t_n, y_n), \quad (26)$$

$$k_2 = f(t_n + h/2, y_n + hk_1/2), \quad (27)$$

$$k_3 = f(t_n + h/2, y_n + hk_2/2), \quad (28)$$

$$k_4 = f(t_n + h, y_n + hk_3), \quad (29)$$

$$y_{n+1} = y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4). \quad (30)$$

The supplied engine applies this to $y = (\rho, \varphi)$ with an autonomous synthetic field, m substeps and $h = \Delta t/m$. The general formal notation allows time dependence; the demonstration coefficients do not vary with time. [S1, pp. 14–17] [C09]

For a sufficiently smooth field and a stable finite interval, Taylor matching through fourth order gives local truncation error $O(h^5)$ and accumulated global error $O(h^4)$. This is not zero drift, exact integration, a quantum-state collapse, or a guarantee of conservation for every field. The LUT introduces its own approximation and its piecewise sampling must not inherit a smooth-field convergence theorem without qualification.

8.1 Separate error terms

A practical comparison must track integration, LUT, coordinate quantization, floating-point and boundary-detection errors separately:

$$E_{\text{total}} \lesssim E_{\text{integration}} + E_{\text{LUT}} + E_{\text{quantization}} + E_{\text{floating}} + E_{\text{boundary}}. \quad (31)$$

This is an error-budget organization, not a universal certified bound for a discontinuous hybrid system. Nearest-angle lookup has angular error at most π/N_φ in exact arithmetic; since sine and cosine are 1-Lipschitz, each coefficient error is at most that amount. Annular drift error also carries the inverse-radius factor and drift coefficients.

8.2 Sampled collision contract

Each child is tested at the hinged start and after every RK4 substep endpoint. Stage evaluations determine the integrator, but they are not swept collision tests. A thin obstacle can be crossed between checked samples and be missed. Reducing h improves sampling but is not, by itself, a continuous-collision proof. Exact event localization or a conservative swept-region method is an explicit future adapter, not an implemented capability.

The CPU/GPU check compares identifiers, active flags, cell indices, statuses and orientation exactly. Finite positions use a tolerance of $3 \times 10^{-5}(1 + |y_{\text{CPU}}|)$. A boundary-cell or status disagreement is still a failure; it is not silently accepted because the positions are close. Floating-point contraction is disabled in the CUDA validation build to reduce one source of arithmetic difference.

9 One-bit word definitions and the POPCNT gate

9.1 Coordinates first, predicates second

Choose N_ρ rows and N_φ angular cells. For a valid canonical coordinate, define

$$i = \left\lfloor N_\rho \frac{\rho - \rho_{\min}}{\rho_{\max} - \rho_{\min}} \right\rfloor, \quad j = \left\lfloor N_\varphi \frac{\varphi}{2\pi} \right\rfloor, \quad (32)$$

$$q = iN_\varphi + j, \quad c_{32} = (i \ll 16) \mid j. \quad (33)$$

The coordinate key c_{32} and linear cell address q are addresses. They are not occupancy bitsets. The environmental bit is obtained by

$$E_q = (M_{\lfloor q/32 \rfloor} \gg (q \bmod 32)) \& 1. \quad (34)$$

Testing `coordinate_word` & `environment_word` has no general spatial meaning. AND is a spatial intersection only after both words use the same cell dictionary. The source's packed-coordinate diagram and mask equation therefore require this address-resolution step. [S1, pp. 16, 25–27, 30–31] [C11]

9.2 Population count and absorption

For same-domain occupancy words A, M , `popcount`($A \wedge M$) counts their shared set bits. Nonzero means at least one common occupied cell. Testing equality to 1 would miss masks whose nonzero bit occurs elsewhere or whose overlap contains multiple bits. The GPU literal word primitive uses CUDA's population count; the per-node propagation path only needs the resolved single membership bit.

A blocked sample sets $a = 0$ and $\sigma = \text{ObstacleSink}$. An out-of-range annular sample sets $a = 0$ and $\sigma = \text{RadialSink}$. This is the Vantablack nullifier's executable meaning. The lineage and reason remain in the journal even though active state is zero. No code erases a person, a physical object or an external account. [C12]

9.3 Space and measure

For $M = N_\rho N_\varphi$ cells, the mask uses $4\lceil M/32 \rceil$ bytes. At 128×1024 , that is 16 KiB, not one 32-bit word for an arbitrarily detailed environment. The angle table uses another 8 KiB. [C11]

A popcount fraction is a fraction of *cells*, not generally a physical area fraction. In the annulus, row-cell area is

$$w_i = \frac{r_*^2}{2} \left(e^{2\rho_{i+1}} - e^{2\rho_i} \right) \Delta\varphi. \quad (35)$$

A physical area estimate must weight occupied cells by w_i . The source's density and saturation language is meaningful only after this choice of measure is explicit.

10 Complete operator-precedence transition

The abstract transition is a composition, not a reliance on C/C++ precedence rules. For each valid live parent and side b , execute

$$\mathcal{P} = \text{Record} \circ \text{AcceptOrSink} \circ \text{SampleMask} \circ \text{RK4AndNormalize} \circ \text{Hinge}_\phi \circ \text{XOR}_J \circ \text{Branch} . \quad (36)$$

Composition is read right to left. A mask is also checked at the hinged start, before the first substep. This resolves the source's before/after-integration variants by making both sampled checkpoints explicit. [S1, pp. 9, 17, 26, 30–31]

```

for each live parent in stable frontier order:
  for side in {0, 1}:
    require parent ID can be shifted without overflow
    child.ID = (parent.ID << 1) | side           # tier 1
    child.active = 1 XOR jitter(tick, child.ID) # tier 2
    if child.active == 0: record JitterVeto; continue
    apply signed lower-case phi hinge           # tier 3
    normalize chart, then test occupancy
    for each of m RK4 substeps while still valid:
      integrate the explicitly supplied field
      normalize annulus/Klein coordinates
      test occupancy at this endpoint
    accept live child OR set active=0 with reason
    record candidate; preserve identity and outcome
compact accepted children into the next frontier

```

10.1 Implementation ownership and scheduling

Candidate $k = 2p + b$ owns output slot k . One GPU thread computes one child; no two threads write the same node. The immutable environment and angle tables are reused. The host supplies fresh packed jitter for a launch, receives the resulting candidate array, writes records in slot order, and stably compacts active children. That establishes deterministic journal ordering independently of GPU thread completion order.

10.2 Resource and failure behavior

The library rejects more than 2^{20} candidate outputs before a generation and guards lineage overflow before shifting. The demonstration CLI permits at most 18 generations. Invalid dimensions, nonfinite parameters, invalid programs and incompatible table shapes are rejected explicitly. A numerical fault becomes an inactive candidate with a reason; an allocation or CUDA API failure stops execution with an error.

A resource-budget failure does not silently truncate the frontier. The journal on disk is a sequential record, however, not a transactional database: a crash or I/O failure can leave a partial run. Verification checks continuity, event count and the retained expected endpoint. Restoring or publishing checkpoints transactionally would require a separate durable-commit layer.

11 Invariants and provenance

Proposition 11.1 (Lineage uniqueness). *If live parent identifiers are distinct and all satisfy the pre-shift bound, every emitted child has a unique identifier and its parent is recovered by a right shift.*

Proof. If $2h + b = 2h' + b'$ with $b, b' \in \{0, 1\}$, reduction modulo 2 gives $b = b'$, then division by 2 gives $h = h'$. The shift bound ensures integer values are not truncated by the finite representation. Since $\lfloor (2h + b)/2 \rfloor = h$, the parent relation is exact. $\square$

Proposition 11.2 (Absorption is closed under future propagation). *A candidate whose active flag is zero is absent from every later descendant frontier unless a separate, explicit new-root input is introduced.*

Proof. Only accepted active candidates are compacted into the next frontier. The next transition iterates that frontier alone and also rejects an inactive parent. Induction over subsequent generations excludes descendants of the absorbed candidate. The XOR rule is never applied as a resurrection rule to dead state. $\square$

Proposition 11.3 (No output-slot races). *For P input parents and $2P$ candidates, distinct candidate threads write disjoint 32-byte node records.*

Proof. Thread index k writes only `out[k]`. The candidate range is checked before the write. Distinct integer indices designate nonoverlapping records. Tables are read-only for the launch. This is a memory-ownership argument, not a substitute for on-device sanitizer execution. $\square$

11.1 Hash-linked records

The genesis digest binds the root digest and an exact manifest string:

$$\ell_0 = \text{SHA256}(D_G \| d \| \text{UTF8}(\text{manifest})). \quad (37)$$

The manifest includes dimensions, field coefficients, topology, jitter threshold, backend label, and digests of the mask and angle-table bytes. Each event is retained as its exact JSON string, giving

$$\ell_{k+1} = \text{SHA256}(D_E \| \ell_k \| \text{UTF8}(e_k)). \quad (38)$$

Domains D_G and D_E are `atomOS:genesis:v2` and `atomOS:event:v2`, each followed by a zero byte. Table words are serialized little-endian; angle components are serialized by their 32-bit float bit patterns.

A trusted retained head detects changed history relative to that head. An attacker who can replace both the complete journal and its expected digest can construct a new self-consistent chain. Hash linking is therefore not synonymous with authenticated identity, physical truth, undeletability or a distributed consensus ledger. The verifier accepts an independently retained expected head to make this boundary explicit. [C05–C06]

12 The texture-programmed universal profile

The source's shift/XOR propagation loop alone does not specify arbitrary state-dependent memory computation. atomOS-U is the explicit programmable extension: its read-only transition table supplies rules, while each machine owns a mutable binary tape. This extension was introduced separately in S2 and now receives a CUDA implementation. [C13,D01]

Let a binary-tape machine have finite states Q , halt state q_H , tape $T : \mathbb{Z} \rightarrow \mathbb{B}$ with finite nonzero support at every finite time, head $h \in \mathbb{Z}$, and transition function

$$\delta : (Q \setminus \{q_H\}) \times \mathbb{B} \rightarrow Q \times \mathbb{B} \times \{-1, 0, +1\}. \quad (39)$$

For $\delta(q, T(h)) = (q', b, d)$, write b at cell h , set $h' = h + d$, and set state q' . Halt when $q' = q_H$. This is a precise programmable state-transition kernel.

12.1 Instruction encoding

The supplied program is an array indexed by $2q + b$. One 32-bit instruction word is

$$I = (q' \ll 8) \mid ((d + 1) \ll 1) \mid b' \mid 8. \quad (40)$$

Bit 0 is the write symbol; bits 1–2 encode move codes 0, 1, 2 for $-1, 0, +1$; bit 3 marks a valid instruction; bits 4–7 must be zero; bits 8–31 store the next state. Zero words are invalid transitions. The mathematical encoding permits 24-bit next-state values; the supplied file loader bounds the table to 65,536 states.

GPU object	Role
Program texture	Read-only array of instruction words; fetched by current state and read symbol.
Packed tape	Writable global-memory words; 32 Boolean cells per word.
Machine state	Current state, head, status, committed transition count; 16 bytes.
Native kernel	Decodes instruction data, checks bounds, performs write/move/state update.

12.2 Finite-device semantics

Each GPU thread owns an entire separately aligned tape-word range. It executes at most the launch's transition budget, then yields with its machine state intact. Reads, writes and next-head bounds are checked. An out-of-range move produces `TapeExhausted` *before* writing or advancing; an invalid instruction produces `BadProgram`. These statuses are not a mathematical halting answer for an unbounded program.

The supplied unary program scans 1s, writes a 1 at the first blank and halts. A second example flips exactly three bits. Program files change behavior without changing the native CUDA interpreter.

13 Constructive simulation and universality

Theorem 13.1 (Exact finite-prefix simulation). *Fix a valid binary-tape program, initial configuration and s transitions of its abstract execution. If every accessed and next-head cell during that prefix lies within a finite allocated interval, all state indices fit the program table, the step counter does not overflow, and the corresponding program words are loaded unchanged, atomOS-U produces the same tape, head and state after each of those transitions.*

Proof. Initially, encode the abstract state directly, offset the abstract head into the allocated interval, and pack tape bit i at word $\lfloor i/32 \rfloor$, bit $i \bmod 32$. Assume the representations agree at step k . Reading that bit yields the same symbol b . Fetching word $2q + b$ retrieves exactly the encoded triple (q', b', d) . The validity checks pass by hypothesis. Masking and replacing the selected bit changes only the intended tape cell. The implementation then assigns q' , moves the head by d , and increments the count. Therefore the representation agrees at step $k + 1$. Induction gives the claim for all s transitions. $\square$

This proof depends on the loaded values, not on whether an instruction fetch hits or misses the texture cache. Texture and ordinary global-memory backends are semantically interchangeable under the same resource and coherence assumptions.

13.1 From binary programs to universal computation

The unbounded logical version can interpret every finite binary-tape transition table. To relate this to a finite-alphabet tape, encode each symbol in a fixed block of k bits, with the blank symbol encoded as all zeros. Finite control reads a complete block into its finite state, selects the source transition, overwrites the block with the encoded output symbol, and advances to the neighboring block. Because k and the alphabet are fixed for a given program, this is implemented with finitely many additional control states and finitely many binary steps per simulated transition.

Consequently, any deterministic finite-alphabet tape-machine computation can be compiled into the binary transition model. An interpreter whose program input is such a table can realize each of those computations with unbounded logical tape. This is the computational universality attributed to atomOS-U; the constructive finite-prefix theorem specifies exactly what a finite device witnesses.

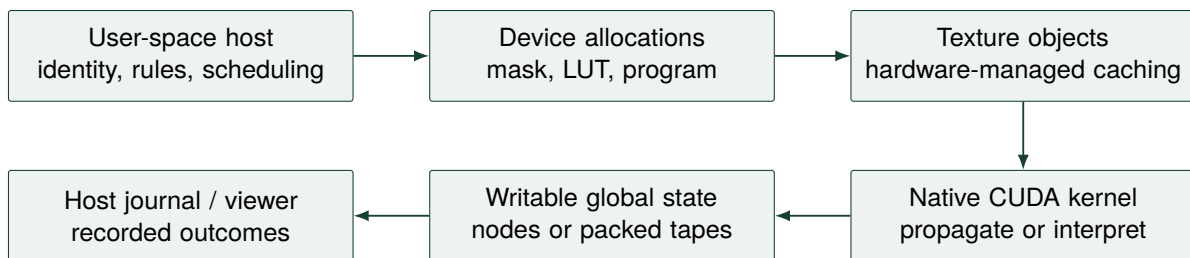
Precise universal-kernel claim. The programmable architecture supports universal computation in the unbounded logical-memory model. A particular RTX 5070 Ti allocation, 32-bit instruction table, 32-bit step counter and finite launch budget remain finite resources. Their exhaustion is reported. Computational universality does not imply prediction of an arbitrary physical future, zero execution time, an answer to every halting question, or universality of the original fixed shift/XOR loop alone.

14 RTX 5070 Ti: what is in texture memory

NVIDIA lists the desktop GeForce RTX 5070 Ti at compute capability 12.0. CUDA Toolkit 12.8 added SM_120 compiler support. The build therefore targets 120; the host code checks the selected device's reported major/minor capability before running. The exact installed card, driver and toolkit must still be checked on the owner's machine. [N1–2]

14.1 Program as data; execution as code

The native functions `mitosis_texture` and `universal_texture` are CUDA kernels. Their executable instructions are not installed into a texture cache. Instead, a texture object describes a read-only view of a device allocation. The programmable kernel fetches instruction *words as data* and interprets them; this is the valid operational sense of texture-programmed execution. [N3–4]



14.2 Exact texture configuration used

The backend creates linear-resource texture objects over `cudaMalloc` allocations. Unsigned 32-bit tables use element-type reads; the angle table uses two float components. The kernels use integer `tex1Dfetch` indices. Resource alignment and linear texture size limits are checked. For linear resources the API ignores addressing/filtering modes, so the code computes and bounds every index itself. It does not rely on a texture sampler to implement the Klein seam. [N3]

14.3 Cache residency is not an allocation promise

The persistent objects in this design are the backing allocations and their texture descriptors. Cache lines remain hardware-managed. A program cannot infer that an entire table is permanently resident merely because the table is small or a warm-up was performed. The supplied package makes no residency or speedup claim; the ordinary global-load backend supplies a controlled alternative for measurement. Texture locality can help some accesses, but the performance comparison is workload-dependent. [N4–5]

No tensor-core or ray-tracing-unit use is implied by targeting an RTX card. This source is ordinary CUDA arithmetic, bit operations, texture reads and global-memory writes.

15 GPU memory layout, lifetime and launch design

15.1 Concrete allocation model

The implementation deliberately keeps ownership simple. A node is 32 bytes: a 64-bit lineage ID, two 32-bit floating coordinates, and four 32-bit fields for active state, cell, status and orientation. A VM state is 16 bytes. Compile-time layout assertions guard these sizes.

For P propagation parents, the principal GPU allocations are

$$M_{\text{GPU}} = 32P + 64P + 4 \left\lceil \frac{2P}{32} \right\rceil + 8N_{\varphi} + 4 \left\lceil \frac{N_{\rho}N_{\varphi}}{32} \right\rceil, \quad (41)$$

excluding runtime metadata and allocation overhead. These terms are input nodes, candidate nodes, jitter words, angle pairs and mask words. With the default table dimensions, immutable angle and mask storage together occupy 24 KiB. This is a storage calculation, not a claim that the 24 KiB remains resident in a particular cache.

For B independent machines, L_T tape bits each and Q states, the principal interpreter storage is

$$M_U = 16B + 4B \lceil L_T/32 \rceil + 8Q. \quad (42)$$

The CLI limits total tape allocation to 256 MiB and machine count to 65,536. Threads own disjoint complete tape-word ranges, including any padding bits.

15.2 Snapshot and synchronization contract

No backing allocation is mutated while a kernel reads it through a texture object. The host waits for a launch to complete, destroys a texture object before resizing its storage, uploads the next snapshot, and binds the new view. Texture objects are destroyed before their backing buffers. This avoids the read/write coherence problem of modifying texture-backed memory during the same launch. [N5]

Mutable nodes and tape words are accessed through ordinary pointers. The same transition table is shared read-only among interpreter threads; each thread's tape writes are private. No cross-block synchronization or inter-machine tape sharing is required by this design.

15.3 Performance-oriented alternatives, not silent changes

The propagation launch uses 256 threads per block; the interpreter uses 128. These are starting values, not a measured optimum. The node array is an array of structures for transparent records; a structure-of-arrays redesign, device-side compaction, CUDA graphs, persistent workloads or field-specific interpolation could improve a future version, but would require a new validated implementation contract.

The delivered host performs hashing, frontier compaction and journal output. It is therefore not an all-device scheduler or an end-to-end constant-time engine. Repeated short launches keep the workload bounded without disabling an operating-system GPU watchdog.

16 Ring 0, access authority and deployment boundaries

16.1 Three meanings that must remain separate

A *CUDA kernel* is a device entry-point function. An *operating-system kernel* is privileged host software. The *universal kernel* in this document is a programmable transition core. Sharing the word “kernel” does not make these the same object.

Windows separates user-mode applications from kernel-mode components and gives them different isolation and failure consequences. The application in this package is a user-mode host program using the installed NVIDIA stack. CPU privilege mode is not a selectable texture-memory property. [N7]

Delivered ring-0 answer. No additional privileged driver is necessary for the texture-object API used here, and none is included. A CUDA launch does not elevate the host process to ring 0. A future legitimate driver integration would be a separately specified, platform-specific project; it would not make this texture program permanently cache-resident.

16.2 A legitimate future integration boundary

For a future device-specific OS integration, keep model execution, program validation, limits and journal generation in user space. Any necessary driver should have a narrow interface, explicit permissions and independently reviewed device responsibilities. Windows kernel-driver distribution has signing requirements; the package neither bypasses nor disables those requirements. [N8]

No driver binary, unsigned-driver loader, arbitrary kernel-memory read/write interface, process-hiding routine, injected game component, DMA bypass, anti-cheat bypass or persistent hidden service is delivered. This is not a claim that every privileged driver is improper; it is the exact boundary of this implementation.

16.3 Source identity and external-control assertions

The later source associates author identity with external platforms, neural control and physical actions. The supplied files do not establish such an operational connection, and this package does not create one. Author metadata is recorded as attribution only. No identity token is treated as a master key to infrastructure. [S1, pp. 31–42] [D04]

The source’s Vantablack operation means an absorbing *simulation state* here. The code has no target-acquisition, firing, drone-actuation or physical-enforcement interface. Likewise, the medical/biochemical vocabulary in the final source pages is not used as a diagnosis, treatment mechanism or claim about the author’s nervous system. These are not gaps filled by invention; they are explicit exclusions from the delivered executable contract.

17 Bush mirage, the ring and the JK operator

17.1 The requested edition as a presentation layer

The “bush mirage / C&C RA2 / Tolkien” request is carried into the package as an original foliage-and-ring visualization, not as an injected game modification. The offline HTML viewer loads a generated `frames.json` file. A checkbox changes node glyphs from points to procedural bush silhouettes while leaving identity, coordinates, mask results and journals unchanged. [D03]

No EA/Command & Conquer or Tolkien game assets, artwork, logos, text passages or font files are bundled. “Mirage / Ring” is an edition label and presentation motif, not a claim of endorsement. The viewer performs local file reads selected by the user; it contains no network requests.

17.2 Formal separation of presentation and state

Let S be recorded state and $m \in \{\text{points, mirage}\}$ the view mode. Rendering is a function

$$R_m : S \longrightarrow \text{display primitives.} \quad (43)$$

Changing m changes $R_m(S)$, not S or $\mathcal{K}_\vartheta$. This is the precise “mirage” invariant. The trace is still auditable, and no memory or process is concealed from the operating system. In Klein mode the viewer shows a chosen chart illustration; it is not a faithful 3-D embedding of the quotient surface.

The ring design names a visible boundary/control motif. It is not renamed CPU ring 0. The ordinary play/pause control advances the view through recorded generations; it does not trigger a live simulator, external action or privileged operation.

17.3 JK truth table, without physical extrapolation

For Boolean state Q and inputs J, K , the next state is

$$Q' = (J \wedge \neg Q) \vee (\neg K \wedge Q). \quad (44)$$

Thus $(J, K) = (0, 0)$ holds, $(1, 0)$ sets, $(0, 1)$ clears, and $(1, 1)$ toggles. This is included in the Python reference tests. The Boolean definition does not establish that a physical circuit or a large software system cannot fail, and it does not connect the user’s body to the engine. [S1, pp. 40–42] [C14]

17.4 Retained metaphor registry

“V gestation” names the state-to-observation staging area. “V mirror” names a conditional symmetry relation. “!!man” can denote deliberate local configuration, not external authorization. “Royal Jelly” is recorded only as a source name that could label a future software checkpoint-restoration operation; that restoration API is not implemented and no biochemical effect is asserted. Keeping these names separate from executable claims preserves the source’s vocabulary without making unsupported physical assertions.

18 Complexity, observables and benchmarking

For P_n live parents at generation n and m substeps, the propagation pass evaluates at most $2P_n$ candidates and $4m$ field stages per surviving candidate. With fixed-size lookups and bounded hash input, work per generation is $\Theta(P_n m)$ in the worst case, plus transfer and journal work. With no absorption, $P_n = 2^n P_0$ and the total emitted frontier grows exponentially with depth. Constant-time *local* operations do not make the entire engine $O(1)$. [S1, pp. 20–22, 29–31] [C13]

The universal interpreter performs $O(Bs)$ transition work for B independent machines and per-launch budget s , subject to early stopping. Parallel execution may reduce elapsed time; a single sequential tape’s dependent transitions are not automatically parallelized.

Observable	Precise definition
Candidate throughput	Candidates divided by measured seconds; name the timing boundary.
Survival fraction	Accepted children divided by emitted candidates for a generation.
Branching ratio	P_{n+1}/P_n for $P_n > 0$; a simulation count ratio, not automatically a clinical R-value.
Absorption fraction	Inactive candidates by status divided by emitted candidates.
Cell occupancy fraction	Occupied cell count divided by total cells; area-weight separately when required.
Interpreter rate	Committed machine transitions divided by measured seconds.
Cache behavior	Measured profiler counters for this launch and access path, not inferred residency.

18.1 Two distinct timing boundaries

-repeats R executes one warm-up then times R repetitions of the same input candidate batch with CUDA events and reports the mean event interval. It does not advance R generations. Upload, allocation, CPU SHA, download, stable compaction and journal writing are outside that interval. The reported CPU wall interval includes orchestration after backend construction and some buffered file output; it is not a durable-to-disk latency measurement.

To compare texture and global reads, use the same seed, tables, generation count, block configuration and validation flags, and repeat the comparison. Record card name, driver, toolkit, workload sizes, thermal/power conditions and both isolated-kernel and end-to-end timing. Nsight Compute exposes section discovery, full profiling and per-device metric queries; use the available counters instead of assuming identical metric names on every installation. [N6]

No invented benchmark. This delivery reports no RTX throughput, texture hit rate, permanent-residency result, speedup factor, zero-latency result or superiority over a Cartesian/A* method. The included global backend and validation scripts make those questions measurable on the target machine.

19 Executed validation and pending GPU acceptance

The delivered reports distinguish test cases, assertions and hardware execution. All measurements below were produced from this version's source, not copied from the previous package's counts. Compiler: GCC 14.2.0. Python: 3.13.5. Host: Linux x86-64. `results/validation_summary.json` records the environment and numerical values.

Check	Executed outcome
Portable C++ groups / assertions	26 groups; 25,051 assertions passed.
Python unittest cases	29 passed; no skipped cases in the saved run.
CTest, release CPU build	4 passed.
CTest, address/undefined-behavior sanitizer build	4 passed.
Independent Python/C++ word comparisons	1,000 cases matched.
SHA-256 byte-string comparisons	100 matched, plus standard C++ digest vectors.
Identity-jitter comparisons	300 matched.
Random bounded machine-program traces	600 matched against a dictionary-tape representation.
Journal checks	Continuity verified; altered event and wrong expected head rejected.
CUDA compilation / GPU execution	Not performed; nvcc and accessible NVIDIA hardware unavailable.
GPU sanitizer / profiler / cache measurement	Not performed.

The random-program comparisons exercise state, head, status, committed count and final tape. They are finite implementation checks. The simulation theorem is a separate mathematical argument; neither is substituted for the other.

19.1 Smooth RK4 control

A separate scalar test integrates $y' = y$, $y(0) = 1$ to $t = 1$ using unquantized double-precision arithmetic. It verifies the expected approach to a factor of 16 when the step is halved. This is not a convergence claim for the LUT-plus-mask hybrid system.

Steps	h	Absolute error	Prior error / error
10	0.1000	2.084e-06	—
20	0.0500	1.358e-07	15.348
40	0.0250	8.666e-09	15.670
80	0.0125	5.473e-10	15.834

19.2 On-device acceptance, still to execute

Build the CUDA target, run CPU/GPU comparisons for texture and global propagation and both interpreter backends, then run Compute Sanitizer. Its tools can report device memory-access and race errors; passing the portable host tests is not a substitute. [N10] The package script saves command logs and records failure or missing prerequisites rather than reporting a successful GPU run. Native binaries are not bundled because CUDA compilation was not performed here.

20 Build, run, inspect and reproduce

20.1 Target GPU build

Use a supported 64-bit Windows or Linux environment, CMake 3.22 or later, CUDA Toolkit 12.8 or later with SM_120 support, a supported host compiler, and a driver supporting both the specific card and toolkit. Compiler support and driver/device support are separate requirements; do not infer support for a newly introduced GPU from the generic CUDA-12.x minimum driver alone. [N1–2,11]

```
cmake -S . -B build_gpu -DATOMOS_ENABLE_CUDA=ON \
  -DCMAKE_CUDA_ARCHITECTURES=120 -DCMAKE_BUILD_TYPE=Release
cmake --build build_gpu --config Release --parallel 2
ctest --test-dir build_gpu -C Release --output-on-failure

./build_gpu/atomos_gpu --backend texture --generations 12 \
  --verify --out run_texture
./build_gpu/atomos_gpu --backend global --generations 12 \
  --verify --out run_global
./build_gpu/atomos_gpu --mode universal --backend texture \
  --verify --out run_vm
```

For a Visual Studio multi-configuration generator, replace the executable path with `build_gpu\Release\atomos_gpu.exe`. The application does not request elevated host privileges. Normal driver/toolkit installation is a separate platform operation.

20.2 CPU-only build and supplied records

```
cmake -S . -B build -DATOMOS_ENABLE_CUDA=OFF
cmake --build build --config Release --parallel 2
ctest --test-dir build -C Release --output-on-failure
./build/atomos_cpu --generations 12 --verify --out run_results
python scripts/verify_journal.py run_results
```

Set `ATOMOS_PROBE` to the compiled probe executable and `ATOMOS_RUN` to a propagation run before executing the Python suite. Without them, the dependent cross-language/journal tests explicitly skip. The POSIX helper `scripts/build_cpu.sh` sets both and executes the complete check sequence.

20.3 A different texture program and the viewer

```
./build_gpu/atomos_gpu --mode universal \
  --program examples/flip_three.tm --tape 101 --machines 1 \
  --backend texture --verify --out run_custom_vm
python scripts/validate_gpu.py --sanitizer
python scripts/verify_manifest.py
```

Open `viewer/index.html` locally and select a run's `frames.json`. The package contains sample CPU runs. Existing files in an explicitly selected output directory are replaced when rerunning the same demo. No network service or external account is needed.

21 Correction register and implementation boundaries

This register states changes rather than silently overwriting the source. The accompanying CSV includes precise S0/S1 page references for every entry.

ID	Source formulation	Corrected or explicit formulation
C01	P becomes B through hinge	A typed coordinate/basis rotation, not conversion of physical units.
C02	XOR / Hadamard / elementwise blend	Distinct operators; the implementation uses Boolean XOR.
C03	Left/right shifts produce two child pointers	Heap lineage uses $2h$ and $2h + 1$; right shift recovers a parent. No search-order invariant is assumed.
C04	OR and XOR both used as the split merge	Separate literal word profiles, with XOR the explicit default.
C05	$U \cup U$ binds two identity layers	Typed pair $C \times H_{256}$; union with itself is idempotent.
C06	One-bit complete hash / stable angle under mutation	Retain all 256 identity bits. Coordinate/predicate projection is lossy; hash mutation has no guaranteed angle continuity.
C07	$\theta = \varphi$ losslessly collapses 3-D space	A two-coordinate model or a declared geometric restriction; the apex $r = 0$ is excluded from the logarithmic chart.
C08	Klein bottle is an ordinary one-bit signed volume	Orientation-reversing quotient plus independent cell occupancy; parity and tangent transformation are explicit.
C09	RK4 prevents drift and detects exact collisions	Field-dependent numerical approximation with separate LUT, rounding and sampled-boundary errors.
C10	$e^{-\rho}$ necessarily damps physical motion	Coordinate-rate conversion; divergence depends on declared component conventions.
C11	A coordinate word AND a mask proves geometry	Resolve the coordinate to a shared cell dictionary before membership/intersection. Nonzero overlap, not equality to literal 1.
C12	Zero reset destroys all tracked identity	Active state is zero; identity, status and provenance are retained.
C13	Constant lookup gives universal zero-latency prediction	Total work scales with emitted candidates and steps. Programmable read/write memory is an explicit extension; physical prediction is a different claim.
C14	JK / Royal Jelly guarantee physical recovery	JK is a Boolean transition law. No biochemical or neural recovery mechanism is implemented.
D01	GPU implementation unspecified	C++/CUDA source, exact texture data formats, dual load paths, bounded ownership and verification procedure added.
D02	Ring 0 requested	No custom privileged component required or supplied; OS-driver work remains separate.
D03	Bush mirage / game / ring edition	Offline procedural presentation, not stealth or game injection.
D04	External, neural and physical control	Not established or implemented. No operational weapon-control or external identity-authority code.

Package inventory and source provenance

The ZIP contains this PDF and its editable LaTeX; the portable C++ core; CUDA texture/global kernels and launch code; an independent Python reference; unit and cross-language tests; sample transition programs; validation and journal scripts; the offline viewer; and actual CPU result files. It contains no vendor compiler/runtime binaries, font files, game assets, custom privileged driver or purported installed GPU image.

Location	Purpose
include/atomos/core.hpp	Shared bit, geometry, propagation and machine semantics.
cuda/kernels.cu	Native texture/global propagation and interpreter entry points; literal word kernel.
cuda/backend.cu	Resource lifetimes, texture binding, launches and timing.
src/	Host seed, tables, input validation, orchestration and journal.
reference/, tests/	Independent reference and executable checks.
docs/, source/	Formal source, correction map, official references and upload digests.
results/	Completed CPU checks and explicit GPU-not-run status.
viewer/, examples/	Local rendering and editable transition tables.

Supplied-document identifiers

Page references use physical PDF pages, starting at 1. The original uploads are identified below but not republished inside the ZIP, avoiding unnecessary redistribution of unrelated personal details and appended claims.

S0. Earlier supplied transcript, 42 pages.

76a65e98937648b65c6121bdbe8fd2334e7f35ab7f30fcec91e4aeb1e4e862e

S1. Latest supplied transcript, 42 pages.

51a5455b59befbd31b9c2c4f73d96a50186d71605fa8fa338d3339044acdf8c

S2. Earlier formalization, 24 pages.

119fb0589a5f4cacb78bc96d204b3fe7988a95ae9005c9a2d22a53f3ca548b24

Version and integrity

The author field is Tom Klootwijk, with the supplied identifier and date recorded in AUTHORSHIP.json. CITATION.cff identifies version 2.0. The checksum manifest covers packaged files and is checked by scripts/verify_manifest.py; these digests are integrity values, not an author signature. No redistribution license for the author's original material has been selected here; the package records that status rather than inventing a license grant.

Official technical references

External references were checked on 13 September 2026. They establish hardware/API terminology, not that this particular engine was compiled or executed on a GPU. The mathematical claims in this document are derived from the stated definitions; the source-specific vocabulary is traced to S0/S1.

N1. NVIDIA. CUDA GPU Compute Capability.

Desktop GeForce RTX 5070 Ti is listed under compute capability 12.0.

<https://developer.nvidia.com/cuda/gpus>

N2. NVIDIA. CUDA Toolkit 12.8 Release Notes.

The release adds compiler support for SM_120; driver compatibility is separately described.

<https://docs.nvidia.com/cuda/archive/12.8.0/cuda-toolkit-release-notes/>

N3. NVIDIA. CUDA Runtime API: Texture Object Management.

Resource descriptors, linear texture restrictions, element-type reads and object lifecycle. The linked Device Management API supplies the format-specific linear-width query.

https://docs.nvidia.com/cuda/cuda-runtime-api/group__CUDART__TEXTURE__OBJECT.html

N4. NVIDIA. CUDA C++ Programming Guide 12.8.

Device execution, memory spaces and texture access.

<https://docs.nvidia.com/cuda/archive/12.8.0/cuda-c-programming-guide/index.html>

N5. NVIDIA. CUDA C++ Best Practices Guide.

Memory-access patterns, texture read/write coherence and performance considerations.

<https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/>

N6. NVIDIA. Nsight Compute CLI.

Profiling sets, metric discovery and report generation.

<https://docs.nvidia.com/nsight-compute/NsightComputeCli/index.html>

N7. Microsoft Learn. User Mode and Kernel Mode.

Host process isolation and kernel-mode execution consequences.

<https://learn.microsoft.com/en-us/windows-hardware/drivers/gettingstarted/user-mode-and-kernel-mode>

N8. Microsoft Learn. Driver Signing Policy.

<https://learn.microsoft.com/en-us/windows-hardware/drivers/install/kernel-mode-code-signing-policy--windows-vista-and-later->

N9. NIST. FIPS 180-4, Secure Hash Standard.

<https://csrc.nist.gov/pubs/fips/180-4/upd1/final>

N10. NVIDIA. Compute Sanitizer.

<https://docs.nvidia.com/compute-sanitizer/ComputeSanitizer/index.html>

N11. NVIDIA. CUDA Compiler Driver 12.8.

<https://docs.nvidia.com/cuda/archive/12.8.0/cuda-compiler-driver-nvcc/index.html>

atomOS 2.0 / Mirage & Ring

Identity-bound state. Explicit operator precedence. Texture-addressed rules. Bounded execution.